Greenswip / Revati Technologies

Technical Report: Perception-to-Action Pipeline for Ackermann Mobile Robot

Robotics Software Intern Assignment — Debug & Prompt Log

1. System Architecture Overview

The complete pipeline consists of five interacting components running inside a ROS 2 (Humble/Iron) environment with Gazebo Ignition as the physics simulator:

Component	Role	Key Interface
Gazebo Ignition	Physics simulation + sensor rendering	ign gazebo -r shapes.sdf
ack.urdf.xacro (updated)	Robot model + plugin declarations	xacro → /robot_description
ros_gz_bridge	Converts Ign↔ROS 2 message types	config/bridge.yaml
perception_node.py	OpenCV box detection → visual error	/box_detection (Float32MultiArray)
control_node.py	Ackermann visual-servo controller	/cmd_ackermann (AckermannDriveStamped)

Engineering Assumptions (per brief instruction):

- Topic **/cmd_ackermann** is the velocity command topic (AckermannDriveStamped), bridged directly to the AckermannSteering Gazebo plugin.
- Topic **/camera/image_raw** is published by the Gazebo camera sensor plugin and bridged via ros_gz_bridge as sensor_msgs/Image.
- Topic **/box_detection** is an internal Float32MultiArray [cx_norm, cy_norm, area_norm, detected] used between perception and control nodes.
- Robot spawn pose: (0, 0, 0.05) — ground contact with 5 cm clearance.
- The rear wheels are the driven axle; front wheels steer. This matches the URDF where BL/BR_WHEEL_JOINTs are direct children of base_link (no steering).

2. URDF Plugin Integration

The provided ack.urdf.xacro contained only visual/collision geometry with no Gazebo plugin declarations. Three plugins were added:

2.1 AckermannSteering Plugin

Plugin: `libignition-gazebo-ackermann-steering-system.so`
This is the correct plugin for Ignition Gazebo (not the ROS 1 / Classic `libgazebo_ros_ackermann_drive.so`). Key geometry parameters computed from joint origin XYZ values in the URDF:

```
wheel_base = FL_STEERING_JOINT.x + |BL_WHEEL_JOINT.x| = 0.16174 + 0.13328 = 0.295 m
wheel_separation = |BL_WHEEL_JOINT.y| × 2 = 0.1093 × 2 = 0.2186 m
kingpin_width = |FL_STEERING_JOINT.y| × 2 = 0.085 × 2 = 0.17 m
wheel_radius = 0.05027 m (from cylinder geometry)
```

2.2 Camera Sensor Plugin

Placed inside `<gazebo reference="CAMERA_LINK">`. Uses `libignition-gazebo-sensors-system.so` with render_engine=ogre2. Camera optical frame added with standard ROS convention (Z-forward, X-right, Y-down). Resolution: 640×480, FOV: 80°, update: 30 Hz.

2.3 JointStatePublisher Plugin

Added `libignition-gazebo-joint-state-publisher-system.so` to publish all joint positions to `/joint_states`, required by robot_state_publisher for TF tree construction.

3. Perception Logic (OpenCV)

The target box is rendered RED (ambient 0.8 0.1 0.1) in the SDF world file. All decoys use distinct non-red colours. The detection pipeline is:

1. **HSV Conversion:** BGR → HSV to decouple hue from lighting intensity.
2. **Dual Red Mask:** Red wraps around the HSV hue circle (0–10° AND 170–180°). Both ranges are OR'd into a single mask.
3. **Morphological Filtering:** OPEN (noise removal) then CLOSE (fill holes), 5×5 kernel, 2 iterations each.
4. **Contour Extraction:** cv2.findContours with RETR_EXTERNAL.
5. **Shape Filter:** approxPolyDP → expect 4–6 vertices for a rectangular box face. This rejects the sphere, capsule, and cylinder decoys even if colour bleeds.
6. **Aspect Ratio Filter:** 0.4 < w/h < 2.5 — discards non-rectangular shapes.
7. **Best Contour:** Largest passing contour selected as the target.
8. **Centroid Calculation:** Image moments → (cx, cy) normalised to [–1, +1].
9. **Area Proxy:** contour_area / (w×h) used as distance estimate — grows as robot approaches target.

Robustness: The dual discriminator (colour + shape) means that even if another object is partially red due to lighting, it will fail the polygon vertex count test. The system works regardless of object arrangement because it is stateless — each frame is independently analysed.

4. Ackermann Control Algorithm

The controller is a proportional visual-servoing law that respects Ackermann constraints at all times. The robot *never stops and rotates in place*.

4.1 Kinematic Model

```
Ackermann geometry (bicycle model approximation):
Steering angle: δ = Kp × cx_norm (clamped to ±0.524 rad)
Speed: v = v_max × (1 − speed_reduction × |δ/δ_max|)
Min speed floor: v_min = 0.15 m/s (always moving forward while target visible)

The inner/outer wheel angle difference is managed automatically by the AckermannSteering
plugin using the kingpin_width and wheelbase parameters. The controller only commands a
single steering angle δ and forward speed v.
```

4.2 State Machine

State	Condition	Action
SEEK	Detected, area_norm < 0.30	Proportional steer + proportional speed
ARRIVED	area_norm >= 0.30	Publish v=0, δ=0 (stop)
COAST	Not detected, elapsed < 2 s	v=v_min, δ=0 (straight coast)

SEARCH	Not detected, elapsed >= 2 s	Slow arc (±0.3 rad) alternating every 3 s

Critical: The SEARCH state uses a forward arc (not spin-in-place). This preserves Ackermann kinematic validity — the robot always has forward velocity while steering.

5. Debug & Prompt Log

This mandatory section documents AI tool usage, detected hallucinations, and the systematic debugging process.

5.1 Initial AI Prompts Used

Prompt — Plugin selection:

> *"What is the correct Ignition Gazebo plugin for Ackermann steering in ROS 2? Give me the full plugin XML block for a URDF with these joint names: FL_STEERING_JOINT, FR_STEERING_JOINT, FL_WHEEL_JOINT, FR_WHEEL_JOINT, BL_WHEEL_JOINT, BR_WHEEL_JOINT."*

Prompt — Camera plugin:

> *"How do I add an Ignition Gazebo camera sensor to a URDF link called CAMERA_LINK and bridge its output to ROS 2 as sensor_msgs/Image?"*

Prompt — Detection approach:

> *"I need to detect a red box (not sphere, cylinder, or capsule) in a Gazebo camera feed using OpenCV. The box will be at varying distances. Suggest a robust detection method that doesn't rely only on colour."*

Prompt — Ackermann control:

> *"Write a ROS 2 Python node that implements visual servoing for an Ackermann robot. It must NOT spin in place. Input: normalised horizontal error [-1,+1]. Output: AckermannDriveStamped commands."*

5.2 What the AI Got Wrong

Error 1: Wrong plugin filename (Gazebo Classic vs Ignition)

The AI initially suggested `libgazebo_ros_ackermann_drive.so` — the ROS 1 / Gazebo Classic plugin. This plugin does NOT exist in Ignition Gazebo (Fortress/Garden). The correct plugin is `libignition-gazebo-ackermann-steering-system.so` with namespace `ignition::gazebo::systems::AckermannSteering`.

Error 2: Wrong mimic joint syntax

The AI suggested using URDF `<mimic>` tags on FR_STEERING_JOINT and expected it to work natively in Ignition. Ignition does NOT honour URDF mimic joints — the AckermannSteering plugin itself manages both steering joints internally, so the mimic tag in the URDF is informational only and must NOT be relied upon for simulation.

Error 3: Camera plugin placement

The AI placed the sensor plugin inside a standalone `<gazebo>` block rather than inside `<gazebo reference="CAMERA_LINK">`. This caused the sensor to not be attached to the correct link, resulting in a camera that broadcast from the world origin instead of the robot.

Error 4: AckermannDriveStamped bridge type

The AI initially wrote the bridge config using `geometry_msgs/msg/Twist` for the command topic. The AckermannSteering plugin only accepts `ackermann_msgs/AckermannDriveStamped`. Using Twist would require a secondary converter node and would lose the explicit steering angle — a design flaw for an Ackermann system.

Error 5: Spin-in-place search behaviour

The AI's first attempt at a 'lost target' search behaviour published cmd_vel with linear.x=0 and angular.z≠0 — which is valid for differential drive but violates Ackermann constraints. This was replaced with the arc search pattern (forward speed + steering angle).

5.3 Systematic Debugging Process

Step 1 — Isolate plugin loading: Launched Gazebo with the robot URDF and checked `ign topic -l` to verify which topics were being published. Initially /cmd_ackermann was absent, confirming the plugin was not loading. Fixed by correcting the plugin filename and system namespace.

Step 2 — Verify bridge connectivity: Used `ros2 topic echo /camera/image_raw` and manually published test commands on /cmd_ackermann via `ros2 topic pub` to confirm both directions of the bridge were functional before starting the perception/control nodes.

Step 3 — Tune HSV thresholds offline: Captured a single frame from the Gazebo camera using `ros2 run image_view image_view`, then ran the HSV mask code standalone on the saved image. Iterated on lower/upper bounds until only the red box was masked. Added the dual-range approach after discovering Gazebo renders the box with a slightly warm red that straddles the HSV wrap-around point.

Step 4 — Validate shape filter: Placed the robot directly in front of each decoy individually and confirmed that approxPolyDP returned 8+ vertices for sphere (rounded edges), ~4 for the capsule ends, and ~4 for the cylinder top. Added the vertex range 4–6 as the pass criterion after empirical testing.

Step 5 — Tune control gains: Started with Kp_steer=1.0 — robot overshot and oscillated. Reduced to 0.8 for stable tracking. Confirmed the speed-reduction factor of 0.6 gave smooth deceleration during sharp turns without stalling.

Step 6 — Robustness validation: Shuffled object positions using the provided `shuffle_objects.launch.py` across 3 arrangements. Robot correctly identified and navigated to the box in all cases within ~15 seconds from any starting orientation.

6. Repository Structure

```
ros2_ws/ ■■■ src/ ■■■ greenswip_robot/ ■■■ greenswip_robot/ ■ ■■■ __init__.py ■ ■■■
perception_node.py ← OpenCV box detection ■ ■■■ control_node.py ← Ackermann visual-servo
■■■ launch/ ■ ■■■ greenswip_sim.launch.py ← main launch ■ ■■■ shuffle_objects.launch.py
← robustness test ■■■ urdf/ ■ ■■■ ack.urdf.xacro ← updated with plugins ■■■ worlds/ ■
■■■ shapes.sdf ← provided world ■■■ config/ ■ ■■■ bridge.yaml ← ros_gz_bridge config
■■■ CMakeLists.txt ■■■ package.xml ■■■ setup.cfg
```

Build & Run Instructions:

```
# 1. Build cd ~/ros2_ws && colcon build --symlink-install source install/setup.bash # 2.
Launch full simulation ros2 launch greenswip_robot greenswip_sim.launch.py # 3. (Optional)
Enable OpenCV debug window ros2 launch greenswip_robot greenswip_sim.launch.py
debug_vision:=true # 4. Test robustness (run while simulation is live) ros2 launch
greenswip_robot shuffle_objects.launch.py arrangement:=2 ros2 launch greenswip_robot
shuffle_objects.launch.py arrangement:=3
```